

Toan Ly

CS 5137 - Deep Learning

Homework 2: Convolutional Neural Network

Step 1. Data

1) For MNIST dataset, I used DataLoader from PyTorch to load this dataset. There are 2 separate train and test sets when loading. The train set has 60,000 images, and the test set has 10,000 images, with the total of 70,000 images.

2) This dataset is trying to solve image classification problem with 10 classes, handwritten digits from 0 to 9.

3) For the pixel intensity value, its original range after transforming using ToTensor() is [0, 1], I then also applied Normalize() to convert the range into [-1, 1].

For the target label, the minimum value is 0 (digit 0) and the maximum value is 9 (digit 9).

4) Each sample image has a shape of 1 x 28 x 28 in grayscale. When using DNN, the images need to be flattened to feed as inputs (784 input features). When using CNN and its variations such as ResNet and VGG, we can still keep the original dimension as 1 x 28 x 28.

5) This dataset is clean and doesn't have any missing values.

6) The label of this dataset is the correspond digit of the sample image, ranging from 0 to 9.

7) Since this dataset is already splitted into training and testing when loading in the beginning, I then only need to split the training set into training and validation sets with the ratio of 8:2. Therefore, there will be 80% training samples (48,000) and 20% validation samples (12,000).

8) For preprocessing, I only converted the data into Tensor and normalize with mean 0.5 and std 0.5 to transform the pixel intensity range from [0, 1] into [-1, 1]. I didn't use any augmentation.

Step 2. Model

Here I trained DNN, CNN, ResNet18, and VGG16. Below is the model architecture choices with their corresponding highest accuracy in the testing dataset:

Model	Accuracy
DNN	0.980
CNN	0.991
ResNet18	0.993
VGG16	0.995

For DNN, I trained with 3 fully connected layers having 128, 64, and 10 neurons, respectively.

For CNN, I set 2 convolution layers as feature extractor with 16 and 32 filters, respectively. Then the outputs from those conv layers will be then feeded into 2 final fully connected layers with 64 and 10 neurons.

The reason why I chose these particular DNN and CNN setup was they have similar total number of trainable parameters (~100k params) and I wanted to see how they perform compared to each other.

For ResNet18, I followed the same architecture as the paper “[Deep Residual Learning for Image Recognition](#)” suggested. I chose the smallest ResNet since MNIST dataset is simple and larger architectures will cost more computing resources.

For VGG16, the architecture was almost similar to the suggested version. The difference is that regular VGG16 requires the input to be 224 x 224. However, since MNIST image has the shape of 28 x 28, after 4 max pooling layers, the final shape is reduced to 1x1 and can't be reduced any further. Therefore, I removed the last max pooling layers from VGG16 since this architecture has 5 max pooling layers.

Additionally, during training, I used the training batch size of 256, relu activation, batch normalization, and early stopping techniques

Step 3: Objective

Cross-entropy (categorical cross-entropy) is the loss function I used to train the models. It's worth to note that PyTorch CrossEntropyLoss has already handled Softmax under the hood, so I didn't add a final softmax layer for these above architectures.

Step 4: Optimization

For optimization, I used Adam with default hyperparameters since overall, Adam is usually better, making training converging faster, than SGD. SGD would usually require specific hyperparameters setup to perform better than Adam.

Step 5: Model selection

Model	LR: 0.1 (F1)	LR: 0.01 (F1)	LR: 0.001 (F1)
DNN	0.962	0.978	0.980
CNN	0.983	0.991	0.991
ResNet18	0.976	0.991	0.993
VGG16	0.776	0.987	0.995

Beside the given learning rates (0.1 and 0.01), I tried an additional smaller learning rate of 0.001 to help the model a little more stable, converge more smoothly, and avoid oscillation in the later epochs during training. Although this small learning rate makes the training more slower, the models actually achieved higher F1 value.

Moreover, I didn't use any learning rate techniques like learning rate scheduling or warmup since I just wanted to explore how those models perform with simple setup

With the above setup, all CNN variations perform better than DNN, and VGG16 has the highest F1 score, but not outperforms ResNet18 and CNN.

To record model performance, I used F1 with "macro" option since all classes in MNIST are treated equally, or this dataset is balanced. Using "weighted" option will also produce similar results.

To overcome overfit, I used validation set and early stopping to help stop training further when the validation loss starts increasing. To overcome underfit, I made the model architectures more complex by adding additional convolution and fully connected layers, more neurons at each layer.

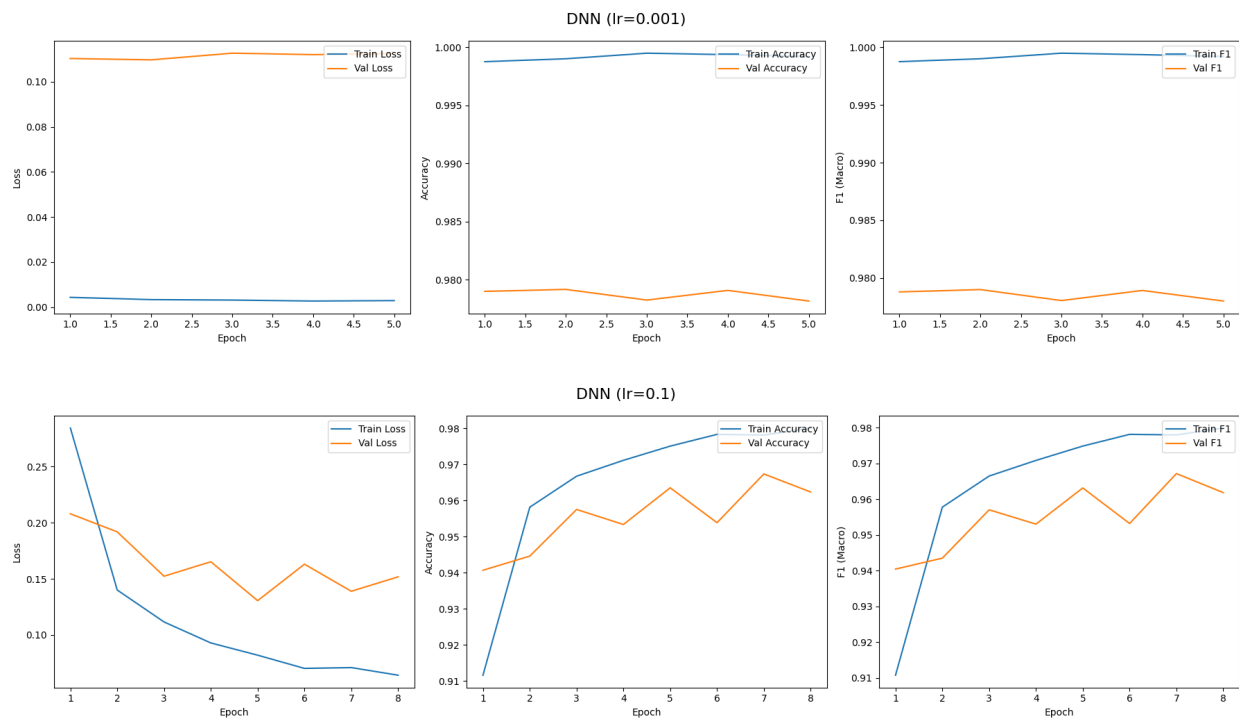
Step 6: Model performance

A.

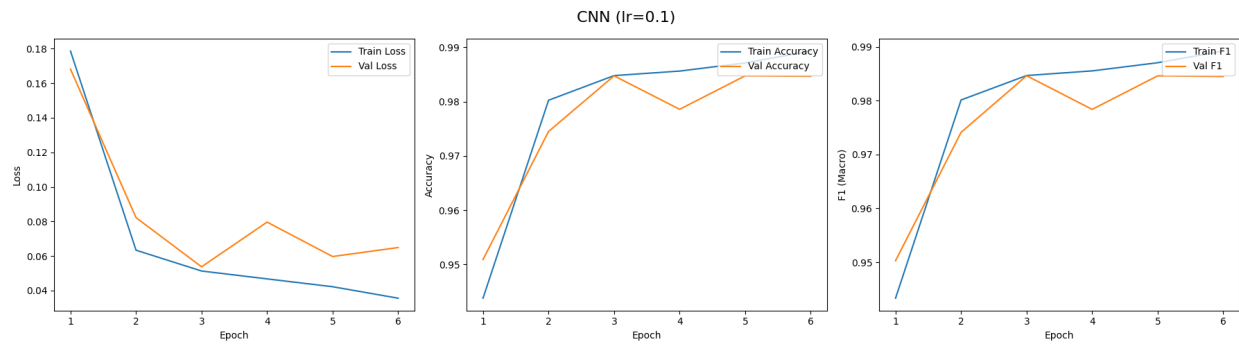
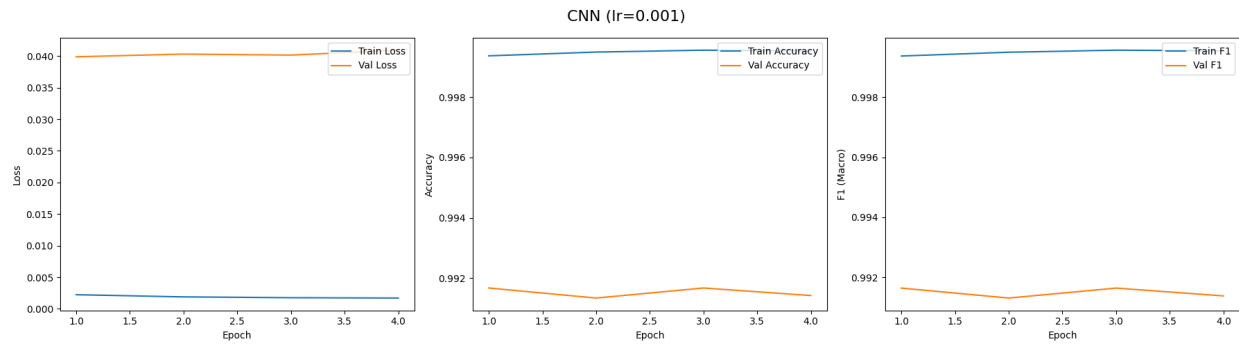
	DNN		CNN		VGG16		ResNet18	
	F1	AUC	F1	AUC	F1	AUC	F1	AUC
LR: 0.1	0.962	0.9989	0.983	0.9998	0.776	0.9946	0.976	0.9997
LR: 0.01	0.978	0.9996	0.991	0.9999	0.987	0.9998	0.991	0.9999
LR: 0.001	0.980	0.9996	0.991	0.9999	0.995	0.9999	0.993	1.0

B. Below are the training plots including training and validation loss, accuracy, and F1 of each best and worst model:

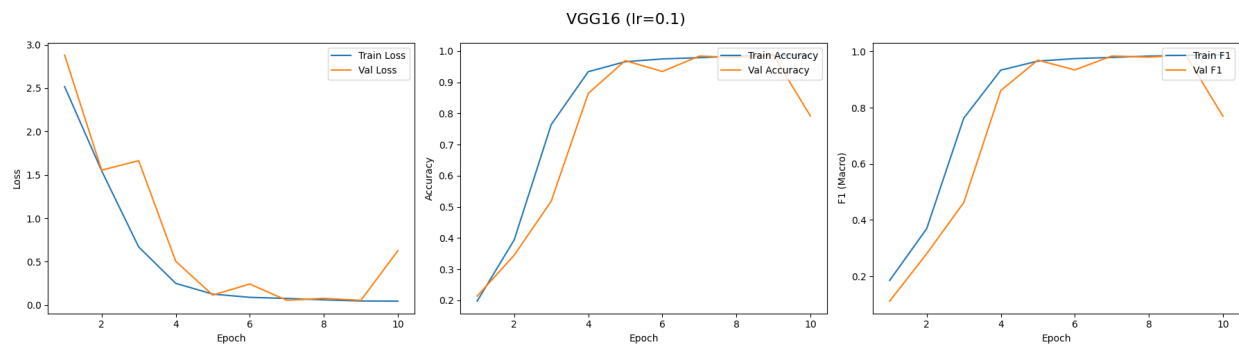
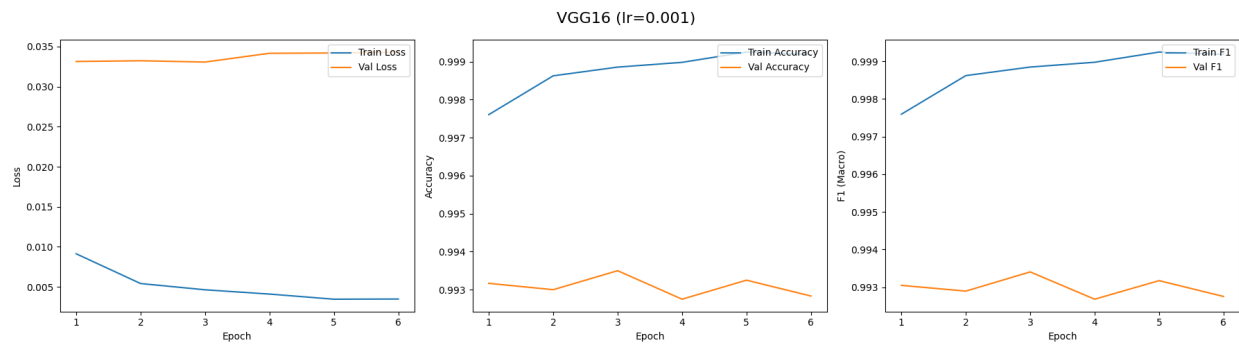
- DNN



- CNN

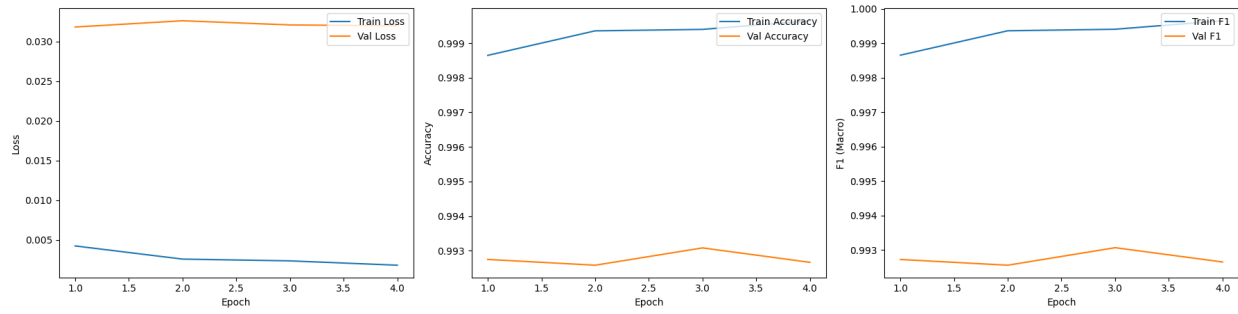


- VGG16

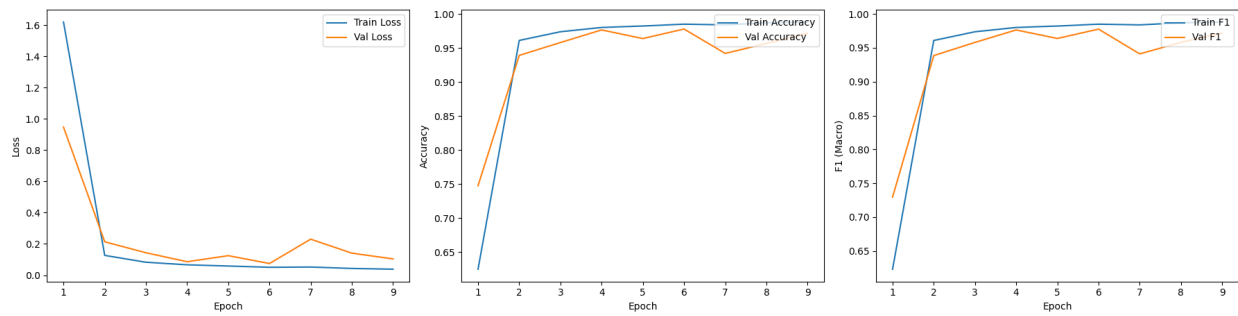


- ResNet18

ResNet18 (lr=0.001)

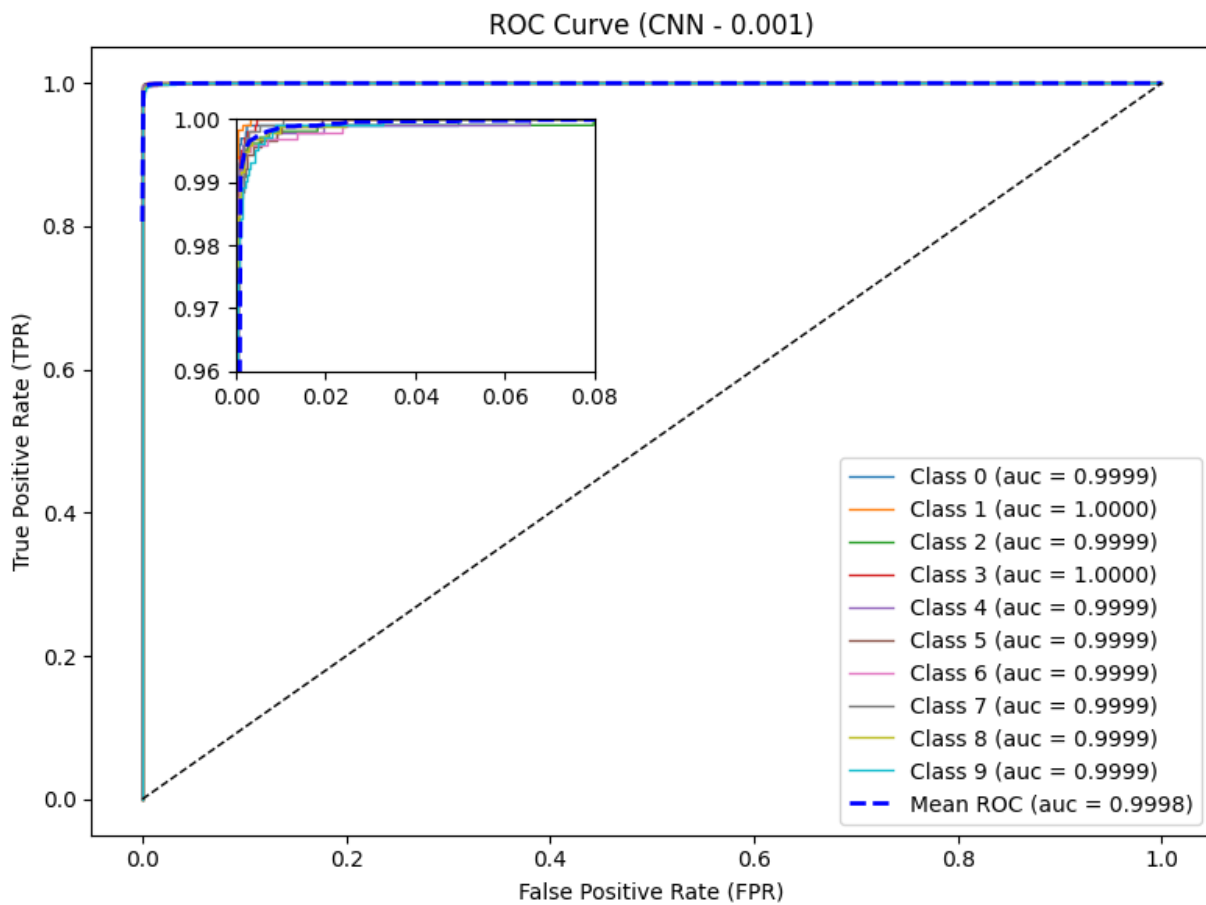


ResNet18 (lr=0.1)

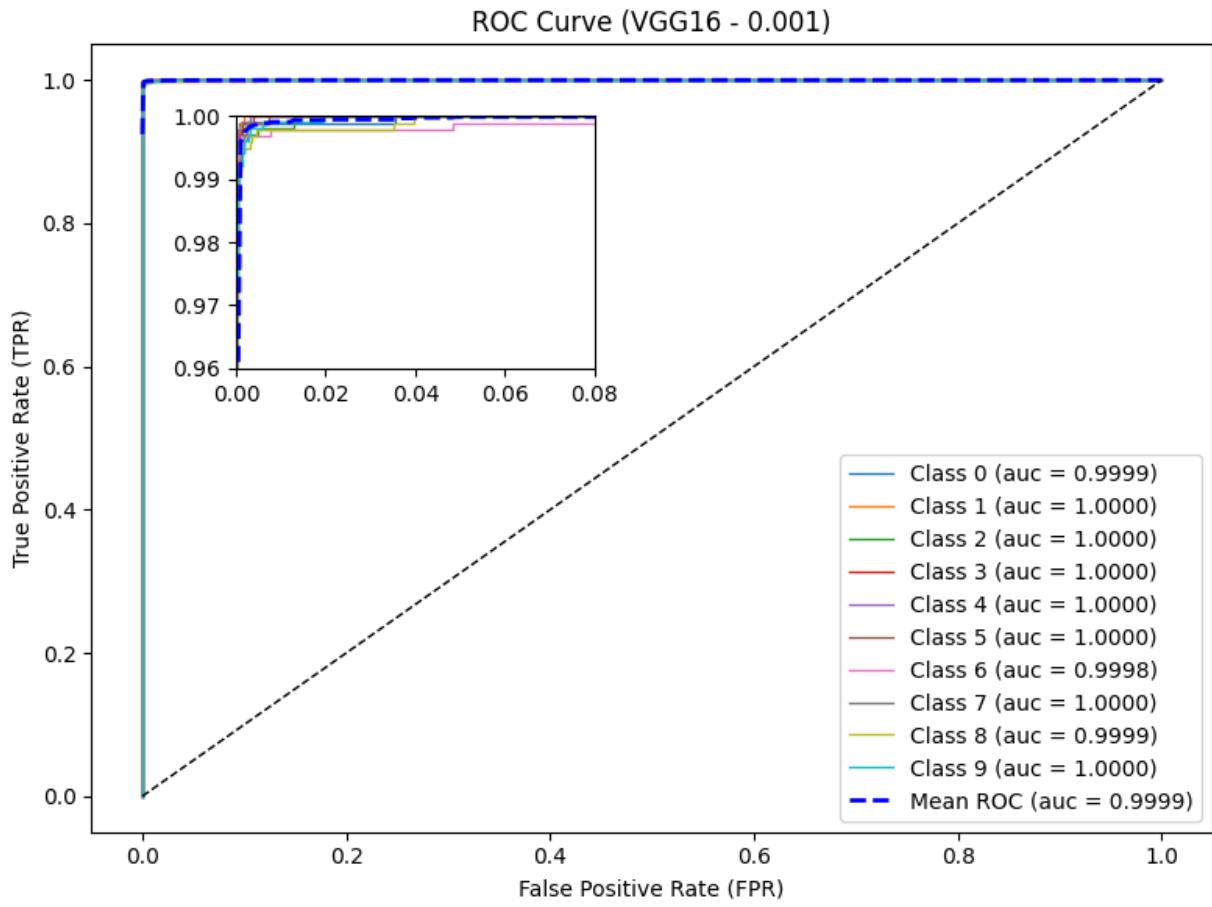


C. Since all 3 CNN versions (CNN, VGG16 and ResNet18) perform almost the same, below are their AUC-ROC curves:

- CNN



- VGG16



- ResNet18

